
Continuación Automática de Playlists con un Modelo Híbrido de Dos Etapas

Agustín Llambías
Pontificia Universidad Católica de Chile

Amaya Quero
Pontificia Universidad Católica de Chile

Larry Uribe
Pontificia Universidad Católica de Chile

IIC3633 Sistemas Recomendadores

Abstract

Este trabajo aborda la tarea de *Automatic Playlist Continuation* (APC): recomendar canciones para extender playlists existentes de manera coherente con su contexto musical. La motivación central es avanzar desde baselines simples, útiles como punto de partida pero limitados por popularidad y baja personalización, hacia modelos que exploten estructura colaborativa y contexto de playlist. Proponemos un sistema híbrido de dos etapas que primero recupera candidatos mediante co-ocurrencia ítem-ítem y luego reordena con señales locales de co-ocurrencia, popularidad, artista y nombre de playlist. Evaluamos el método sobre Spotify Playlists con un protocolo *leave-last-out*, comparándolo en una progresión experimental: Random, Most Popular, Playlist Name Popular, Sequential Markov, Collaborative ItemKNN y variantes ablacionadas del modelo propuesto. Los resultados muestran que el modelo híbrido alcanza un HitRate@10 competitivo frente a Markov, supera a ItemKNN en exactitud y cobertura, y mejora la exploración del catálogo. Las ablaciones confirman que la co-ocurrencia es la señal más importante, mientras que el análisis por segmentos revela una caída importante en objetivos de cola larga. Código, notebooks ejecutados y resultados reproducibles: <https://github.com/larryuc/Proyecto-RecSys-APC-Final>.

1 Introducción

La continuación automática de playlists es un caso central de recomendación musical: dado un conjunto ordenado o semiordenado de canciones, el sistema debe sugerir nuevos ítems que mantengan coherencia con el gusto implícito de la playlist. El problema es relevante porque las playlists concentran preferencias, contexto de escucha y decisiones curatoriales de usuarios sin requerir ratings explícitos. En un catálogo musical de millones de canciones, recomendar bien no consiste solo en proponer canciones populares: el sistema debe inferir afinidad desde una evidencia parcial y, al mismo tiempo, evitar cerrar la experiencia sobre un subconjunto pequeño del catálogo.

La dificultad del problema aparece desde los baselines. Un recomendador aleatorio funciona como control mínimo, Most Popular captura sesgo global del catálogo y Playlist Name Popular agrega una primera señal contextual basada en texto. Sin embargo, estos modelos no capturan suficientemente relaciones entre canciones que suelen aparecer juntas. Por ello, la investigación avanza en forma incremental: primero se establece un piso con baselines simples; luego se incorporan comparadores más fuertes, secuenciales y colaborativos; finalmente se evalúa un modelo híbrido que combina recuperación por co-ocurrencia con reranking multi-señal.

Nuestro objetivo es construir una solución reproducible y local para APC, sin depender de APIs externas durante la evaluación final. La pregunta de investigación es: *¿qué tan competitivo es un sistema híbrido de dos etapas frente a métodos secuenciales y colaborativos, considerando métricas de accuracy y beyond-accuracy?*

Las contribuciones del trabajo son tres. Primero, se define un protocolo reproducible de evaluación offline para APC sobre Spotify Playlists, con muestreo determinístico y split *leave-last-out*. Segundo, se compara una escalera de modelos que parte en baselines simples y termina en un modelo híbrido de dos etapas, permitiendo atribuir mejoras a componentes concretos. Tercero, se reportan resultados no solo en accuracy, sino también en cobertura, novedad, diversidad, robustez entre muestras y desempeño por segmentos.

2 Trabajo relacionado

La recomendación musical ha sido estudiada desde enfoques colaborativos, basados en contenido y modelos híbridos. En escenarios con feedback implícito, como playlists, la presencia de una canción en una lista se interpreta como señal positiva de afinidad. Esto permite construir métodos item-item, donde dos canciones se consideran cercanas si co-aparecen frecuentemente en contextos similares. Esta familia de modelos es simple, interpretable y eficiente para catálogos grandes, por lo que constituye un punto de comparación natural para APC.

La literatura de APC también destaca la importancia del orden y del contexto local. Un modelo Markov de primer orden captura transiciones entre canciones y puede ser competitivo cuando la última canción de la playlist contiene información fuerte sobre la continuación probable. En paralelo, métodos híbridos combinan señales colaborativas y de contenido para enfrentar sparsity, popularidad y cola larga. Nuestro enfoque se ubica en esta familia: usa co-ocurrencia como señal estructural principal y agrega señales livianas de popularidad, artista y nombre de playlist para refinar el ranking.

Finalmente, la evaluación de recomendadores musicales no debe reducirse a una única métrica de accuracy. Dos modelos con desempeño parecido en HitRate pueden diferir mucho en cobertura de catálogo, novedad o diversidad intra-lista. Por esta razón, además de HitRate@10, Precision@10, MAP@10 y nDCG@10, reportamos ILD@10, Novelty@10 y CatalogCoverage@10. Esta decisión es importante para discutir si el sistema solo acierta canciones frecuentes o si también permite una exploración más amplia del catálogo.

3 Dataset

Usamos el dataset Spotify Playlists de Kaggle [1]. El archivo original contiene aproximadamente 12.2 millones de interacciones y pesa 1.18 GB, por lo que no se incluye en el repositorio final. El código espera el archivo en `data/spotify_dataset.csv`. Para mantener reproducibilidad, la muestra se obtiene por hashing determinístico de `user_id || playlistname`, preservando playlists completas.

La ejecución principal usa una muestra del 30% y evalúa 63,851 playlists con un catálogo de entrenamiento de 1,226,786 ítems. Como prueba de robustez, repetimos el protocolo con una muestra del 20%, evaluando 28,065 playlists y 696,943 ítems de entrenamiento.

Formalmente, cada fila del dataset se interpreta como una interacción implícita positiva (u, p, i) entre usuario u , playlist p e ítem musical i . Definimos cada ítem como el par normalizado artista-canción, lo que reduce ambigüedad respecto a usar solo nombre de track. Dado que no existen ratings ni timestamps confiables, el orden observado dentro de cada playlist se usa únicamente para construir el split *leave-last-out*; no asumimos que represente una secuencia temporal estricta. Esta decisión vuelve el protocolo exigente: para cada playlist se debe recuperar un único target dentro de un catálogo de más de un millón de candidatos.

El muestreo por hash evita seleccionar canciones individuales de forma independiente, lo que rompería la estructura de playlists y contaminaría el contexto de evaluación. Al preservar playlists completas, cada ejemplo mantiene la relación entre nombre de playlist, conjunto de canciones y target oculto. Además, la repetición con 20% permite separar conclusiones robustas de variaciones atribuibles al tamaño de muestra.

4 Metodología

El protocolo experimental es *leave-last-out*: para cada playlist con al menos tres canciones, se oculta la última canción como target y el modelo recibe el resto como contexto. Recomendamos Top-10 y excluimos canciones ya vistas en la playlist.

La evaluación se realiza en dos fases. Primero se construyen estructuras de entrenamiento usando solo los contextos C_u , evitando que el target y_u participe en conteos de co-ocurrencia, popularidad o transiciones. Luego, cada modelo genera recomendaciones sobre $\mathcal{I} \setminus C_u$. Esta separación es clave para evitar fuga de información: si el ítem retenido permaneciera en los índices de entrenamiento de su misma playlist, los modelos de co-ocurrencia podrían sobreestimar su desempeño.

4.1 Progresión experimental desde baselines

La evaluación se organiza como una progresión de dificultad. El primer nivel contiene Random y Most Popular, que permiten cuantificar el rendimiento mínimo y el efecto de popularidad global. El segundo nivel incorpora Playlist Name Popular, baseline específico al dominio que usa tokens del nombre de la playlist y representa la primera mejora contextual respecto a popularidad pura. El tercer nivel agrega comparadores fuertes: Sequential Markov, que explota continuidad local, y Collaborative ItemKNN, que explota similitud ítem-ítem por co-ocurrencia. El último nivel corresponde al modelo propuesto, Two-Stage Hybrid, junto con ablaciones que remueven cada señal para medir su aporte.

Esta organización responde a la narrativa experimental del proyecto: no basta con mostrar un modelo final aislado, sino que se debe explicar qué problema resuelve cada nueva familia de métodos y qué mejora concreta aporta frente al punto de partida.

4.2 Modelo propuesto

El modelo **Two-Stage Hybrid** tiene dos etapas. Primero, recupera candidatos usando vecinos ítem-ítem por co-ocurrencia normalizada. Luego reordena esos candidatos con una combinación ponderada de cuatro señales: co-ocurrencia, popularidad, coincidencia de artista y tokens del nombre de la playlist. La señal cooc corresponde a la frecuencia relativa con que dos canciones aparecen dentro de las mismas playlists; al removerla, el sistema debe recurrir a candidatos basados en popularidad, artista y texto.

La motivación de la arquitectura de dos etapas es computacional y metodológica. Computacionalmente, puntuar todo el catálogo para cada playlist es costoso; por eso el retrieval reduce el espacio de búsqueda a candidatos plausibles conectados con el contexto. Metodológicamente, separar retrieval y reranking permite distinguir entre *generar* opciones musicalmente cercanas y *ordenarlas* según señales adicionales. La primera etapa prioriza recall de candidatos, mientras que la segunda intenta mejorar precisión temprana en el top-10.

El score de artista $s_{\text{artist}}(j, u)$ aumenta cuando el artista del candidato aparece en el contexto de la playlist, capturando continuidad estilística local. El score de nombre $s_{\text{name}}(j, u)$ aprovecha tokens como géneros, décadas o estados de ánimo presentes en títulos de playlists. La popularidad suaviza casos de baja evidencia colaborativa, pero su peso se mantiene acotado para no transformar el sistema en Most Popular. Esta combinación busca un equilibrio: co-ocurrencia aporta estructura, artista y nombre aportan contexto, y popularidad aporta estabilidad.

4.3 Baselines y comparadores

Comparamos contra tres baselines iniciales y dos comparadores fuertes. Sea $P_u = (i_1, \dots, i_m)$ una playlist observada, $C_u = \{i_1, \dots, i_{m-1}\}$ su contexto de entrenamiento, $y_u = i_m$ el ítem retenido para evaluación y $\mathcal{I}$ el catálogo de canciones candidatas. Todos los modelos producen un ranking $R_u @ K = (r_1, \dots, r_K)$ con $r_j \notin C_u$.

Random recomienda ítems uniformemente desde $\mathcal{I} \setminus C_u$. No busca ser competitivo, sino entregar un control inferior para verificar que el protocolo no genera aciertos artificiales. **Most Popular** ordena candidatos por frecuencia global de entrenamiento:

$$s_{\text{pop}}(i) = \log(1 + \text{freq}(i)), \quad (1)$$

donde $\text{freq}(i)$ es el número de apariciones del ítem i en playlists de entrenamiento. Este baseline captura el sesgo de popularidad del catálogo, pero ignora por completo el contexto particular de cada playlist.

Playlist Name Popular introduce la primera señal contextual. Se tokeniza el nombre de la playlist y se acumulan conteos de ítems observados en playlists con tokens similares. Para un conjunto de tokens T_u , el score se aproxima como:

$$s_{\text{name}}(i, u) = \sum_{t \in T_u} \frac{\text{count}(i, t)}{\text{rank}_t(i) + 1}, \quad (2)$$

donde $\text{count}(i, t)$ resume la asociación entre el token t y el ítem i , y $\text{rank}_t(i)$ favorece canciones más frecuentes dentro de ese token. Este modelo es importante narrativamente porque muestra cuánto se gana al pasar de popularidad global a una señal específica de playlist.

Luego se incorporan comparadores más exigentes. **Sequential Markov** estima transiciones de primer orden entre canciones consecutivas y puntúa candidatos según la última canción observada:

$$s_{\text{markov}}(j \mid i_{m-1}) = \frac{\text{trans}(i_{m-1}, j)}{\sum_{\ell \in \mathcal{I}} \text{trans}(i_{m-1}, \ell)}. \quad (3)$$

Este modelo captura continuidad local y por eso es un rival fuerte en MAP y nDCG. **Collaborative ItemKNN** construye vecinos por co-ocurrencia entre ítems. La similitud usada en retrieval se normaliza como:

$$s_{\text{cooc}}(i, j) = \frac{\text{cooc}(i, j)}{\sqrt{\text{freq}(i)\text{freq}(j)}}. \quad (4)$$

El score de un candidato j para la playlist u agrega evidencia desde los ítems del contexto:

$$s_{\text{knn}}(j, u) = \sum_{i \in C_u} s_{\text{cooc}}(i, j). \quad (5)$$

Finalmente, el modelo **Two-Stage Hybrid** usa esta misma familia de vecinos para recuperar candidatos, pero aplica un reranking lineal:

$$s_{\text{hybrid}}(j, u) = 0.55s_{\text{cooc}}(j, u) + 0.15s_{\text{pop}}(j) + 0.15s_{\text{artist}}(j, u) + 0.15s_{\text{name}}(j, u). \quad (6)$$

Las ablaciones remueven uno de estos términos y renormalizan la comparación, lo que permite atribuir caídas de desempeño a señales concretas.

4.4 Métricas

Reportamos métricas de ranking y métricas beyond-accuracy. Como el protocolo *leave-last-out* tiene un único ítem relevante por playlist, definimos $\text{rel}_u(r) = \mathbb{I}[r = y_u]$ para el ranking $R_u @ K$. Para N playlists evaluadas y $K = 10$, $\text{HitRate}@K$ mide si el target aparece al menos una vez en el top-K:

$$\text{HitRate}@K = \frac{1}{N} \sum_{u=1}^N \mathbb{I}[y_u \in R_u @ K]. \quad (7)$$

$\text{Precision}@K$ se define como la fracción de recomendaciones relevantes en el top-K:

$$\text{Precision}@K = \frac{1}{N} \sum_{u=1}^N \frac{1}{K} \sum_{r \in R_u @ K} \text{rel}_u(r). \quad (8)$$

Dado que existe un único target, la average precision de una playlist equivale al inverso de la posición del acierto:

$$\text{AP}@K(u) = \begin{cases} \frac{1}{\text{rank}_u(y_u)} & \text{si } y_u \in R_u @ K, \\ 0 & \text{en otro caso,} \end{cases} \quad \text{MAP}@K = \frac{1}{N} \sum_{u=1}^N \text{AP}@K(u). \quad (9)$$

nDCG@K también premia posiciones altas, pero con descuento logarítmico:

$$\text{nDCG}@K(u) = \begin{cases} \frac{1}{\log_2(\text{rank}_u(y_u)+1)} & \text{si } y_u \in R_u @ K, \\ 0 & \text{en otro caso.} \end{cases} \quad (10)$$

Table 1: Resultados principales de la ejecución 30%. HitRate, Precision, MAP, nDCG y cobertura se reportan en porcentaje; ILD y Novelty se reportan en escala original.

Modelo	HitRate@10	Precision@10	MAP@10	nDCG@10	ILD@10	Novelty@10	Coverage@10
Sequential Markov	12.57	1.26	8.13	9.19	0.975	16.570	9.10
Two-Stage Hybrid	12.54	1.25	6.84	8.17	0.615	18.847	15.66
Ablation sin pop	12.39	1.24	6.79	8.10	0.619	19.139	16.20
Ablation sin name	11.76	1.18	6.31	7.58	0.637	18.861	15.27
Ablation sin artist	11.43	1.14	6.19	7.40	0.677	18.980	15.41
Collaborative ItemKNN	9.66	0.97	5.51	6.47	0.739	19.041	14.47
Ablation sin cooc	3.87	0.39	1.62	2.14	0.539	16.091	7.40
Playlist Name Popular	1.23	0.12	0.46	0.64	0.672	16.537	6.23
Most Popular	0.11	0.01	0.01	0.04	1.000	12.863	0.00
Random	0.00	0.00	0.00	0.00	1.000	20.741	40.57

Para beyond-accuracy usamos tres métricas. La diversidad intra-lista se aproxima mediante diferencia de artistas entre pares recomendados:

$$\text{ILD@K}(u) = \frac{2}{K(K-1)} \sum_{a < b} \mathbb{I}[\text{artist}(r_a) \neq \text{artist}(r_b)]. \quad (11)$$

La novedad usa auto-información basada en popularidad de entrenamiento:

$$\text{Novelty@K}(u) = \frac{1}{K} \sum_{r \in R_u @ K} -\log_2 \left(\frac{\text{freq}(r) + 1}{|\mathcal{D}_{train}| + |\mathcal{I}|} \right). \quad (12)$$

Finalmente, la cobertura de catálogo mide qué proporción del catálogo aparece al menos una vez en las recomendaciones:

$$\text{Coverage@K} = \frac{\left| \bigcup_{u=1}^N R_u @ K \right|}{|\mathcal{I}|}. \quad (13)$$

En las figuras y tablas de robustez, el término score no representa una métrica adicional: corresponde al valor de la métrica específica graficada o tabulada.

5 Análisis de parámetros

El sistema final usa una configuración fija para hacer comparables las ejecuciones: Top-K=10, split *leave-last-out*, hasta 80 ítems por playlist para construir co-ocurrencias, 150 vecinos máximos por semilla y 400 candidatos máximos antes del reranking. El reranker del modelo Two-Stage Hybrid pondera co-ocurrencia con peso 0.55 y distribuye el resto entre popularidad, coincidencia de artista y tokens del nombre de playlist con peso 0.15 cada uno. La señal LastFM se dejó en peso 0.0 en la ejecución principal por reproducibilidad y latencia de API.

Además del ajuste fijo, evaluamos la sensibilidad de parámetros mediante dos mecanismos. Primero, repetimos el experimento con una muestra de 20% para verificar robustez del orden relativo de los modelos. Segundo, realizamos ablaciones removiendo co-ocurrencia, popularidad, artista o nombre de playlist. Esta evaluación muestra que la co-ocurrencia es el parámetro/señal más determinante del sistema, mientras que artista y nombre aportan mejoras incrementales y popularidad cumple un rol más acotado de estabilización.

Los límites de 80 ítems por playlist y 150 vecinos por semilla controlan el crecimiento cuadrático de pares de co-ocurrencia en playlists largas. Sin esta poda, listas muy extensas dominarían el índice y aumentarían memoria sin necesariamente mejorar la calidad de ranking. El máximo de 400 candidatos fuerza al reranker a operar sobre un conjunto suficientemente diverso, pero todavía manejable. Finalmente, los pesos del reranker se fijan antes de reportar resultados para evitar sobreajuste al conjunto de evaluación; por ello, las ablaciones se interpretan como análisis causal aproximado de señales, no como búsqueda exhaustiva de hiperparámetros.

6 Resultados

La Tabla 1 resume la progresión completa. El primer hallazgo es que los baselines simples son insuficientes para APC a esta escala: Random no logra aciertos relevantes, Most Popular casi no

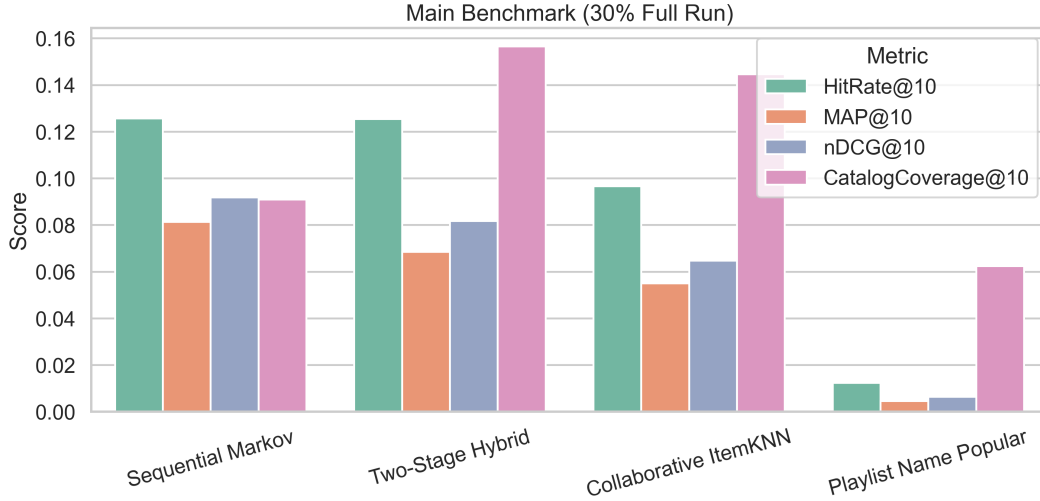


Figure 1: Benchmark principal de la ejecución 30%. Se comparan métricas de ranking y cobertura para los modelos base, comparadores fuertes y el modelo híbrido propuesto.

mejora y Playlist Name Popular, aunque incorpora contexto textual, permanece muy por debajo de los métodos colaborativos y secuenciales. Esto valida la necesidad de avanzar más allá de popularidad y texto superficial.

El segundo hallazgo es que los comparadores fuertes capturan señales complementarias. Sequential Markov lidera en MAP@10 y nDCG@10, lo que indica que modelar transiciones locales favorece la posición del target cuando existe continuidad fuerte al final de la playlist. Collaborative ItemKNN mejora sustancialmente sobre los baselines, pero queda por debajo del modelo híbrido en HitRate@10, MAP@10, nDCG@10 y cobertura. En este contexto, Two-Stage Hybrid alcanza un HitRate@10 prácticamente equivalente a Markov y aumenta la cobertura de catálogo. La mejora no es una victoria absoluta en todas las métricas, sino un trade-off favorable: se sacrifica parte del ordenamiento fino de Markov, pero se gana exploración y cobertura sin perder competitividad en aciertos.

También es relevante separar accuracy de cobertura. Random alcanza alta cobertura porque dispersa recomendaciones por el catálogo, pero no acierta. Most Popular obtiene cobertura casi nula porque recomienda repetidamente los mismos ítems frecuentes. El modelo híbrido se ubica en un punto intermedio más útil: mejora fuertemente la cobertura respecto de Markov y mantiene accuracy competitiva. Esta lectura justifica reportar métricas beyond-accuracy junto a métricas de ranking.

Desde una perspectiva de experiencia de usuario, esta diferencia es importante porque una playlist extendida no debería converger siempre al mismo conjunto de canciones populares. El modelo híbrido no introduce aleatoriedad estocástica explícita, pero sí aumenta la variedad efectiva de recomendaciones: alcanza una cobertura de catálogo de 15.66%, frente a 9.10% de Sequential Markov y casi 0% de Most Popular, y mantiene una novedad de 18.85 bits. Esto sugiere una exploración más amplia y controlada del catálogo, evitando que la continuación automática se reduzca a repetir los mismos ítems frecuentes.

6.1 Ablaciones

Las ablaciones permiten transformar la comparación en explicación. Remover co-ocurrencia produce la mayor caída de MAP@10, lo que confirma que es la señal estructural central del sistema. Este resultado es coherente con la tarea: si el objetivo es continuar playlists, las co-apariciones históricas entre canciones son la evidencia más directa de afinidad. Remover artista o nombre de playlist también reduce ranking, aunque con menor magnitud, porque estas señales refinan candidatos ya plausibles. La popularidad tiene impacto pequeño en MAP@10, pero ayuda a estabilizar la cobertura y evita que el reranker dependa exclusivamente de señales escasas.

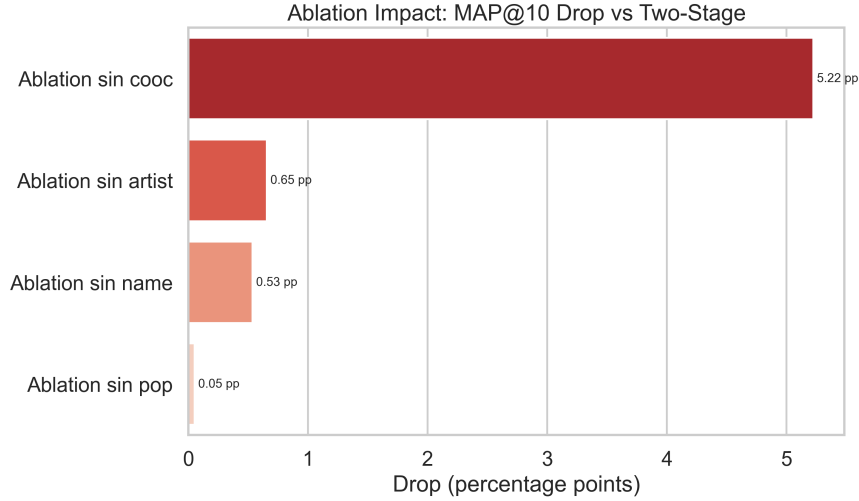


Figure 2: Caída de MAP@10 al remover cada señal del Two-Stage Hybrid.

Table 2: Robustez entre ejecuciones. El score de cada barra/entrada corresponde explícitamente al valor de la métrica indicada: HitRate@10, MAP@10 o nDCG@10. Valores en porcentaje.

Modelo	Ejecución	HitRate@10	MAP@10	nDCG@10	Coverage@10
Sequential Markov	30% full	12.57	8.13	9.19	9.10
Sequential Markov	20% sensibilidad	9.31	6.18	6.93	8.44
Two-Stage Hybrid	30% full	12.54	6.84	8.17	15.66
Two-Stage Hybrid	20% sensibilidad	9.34	4.57	5.68	14.19
Collaborative ItemKNN	30% full	9.66	5.51	6.47	14.47
Collaborative ItemKNN	20% sensibilidad	6.42	3.33	4.05	12.54

6.2 Robustez

La Tabla 2 reporta robustez usando métricas concretas de ranking y cobertura, no un score agregado. La ejecución de sensibilidad reduce los valores absolutos, pero preserva el patrón principal: Markov sigue fuerte en ranking, mientras que Two-Stage Hybrid mantiene mejor cobertura y rendimiento competitivo en HitRate@10. Esto sugiere que las conclusiones no dependen solamente de una muestra puntual del 30%, sino que se sostienen al cambiar la escala de entrenamiento y evaluación.

6.3 Análisis por segmentos

El desempeño depende fuertemente del segmento. Las playlists cortas y medianas con targets populares alcanzan los mayores HitRate@10, mientras que los targets de cola larga en playlists largas son el principal punto débil. Este resultado es consistente con la naturaleza de co-ocurrencia: cuando un ítem raro aparece poco, la evidencia colaborativa disponible es limitada.

La segmentación separa dos fuentes de dificultad. El tamaño de playlist aproxima cuánta evidencia contextual tiene el modelo, mientras que la popularidad del target aproxima cuánta evidencia histórica existe para recuperar ese ítem. Los resultados muestran que más contexto no siempre implica mejor predicción: en playlists largas con targets de cola larga, el contexto puede ser heterogéneo y el target puede tener pocos vecinos confiables. En cambio, playlists cortas o medianas con targets head concentran señales más densas y producen los mejores resultados. Esto sugiere que futuras mejoras deberían ser adaptativas: no basta con un único conjunto de pesos para todos los segmentos.

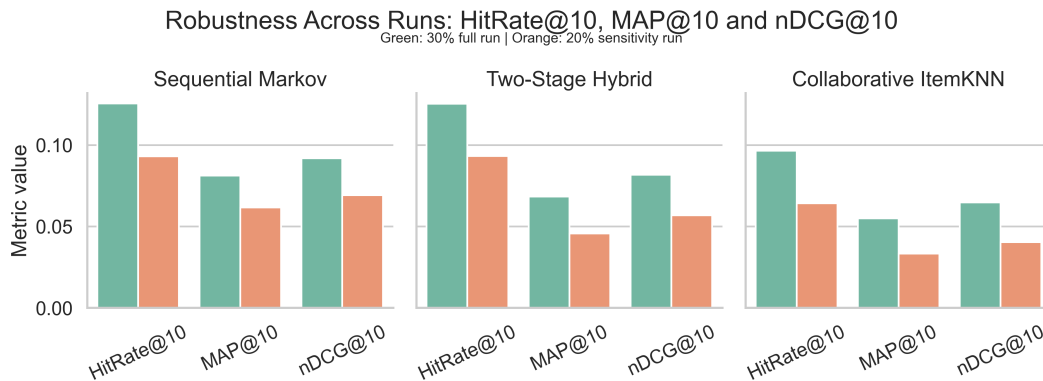


Figure 3: Robustez entre ejecuciones. El eje vertical muestra el valor de la métrica indicada en cada panel: HitRate@10, MAP@10 o nDCG@10.

Table 3: Desempeño del modelo Two-Stage Hybrid por segmento en la ejecución 30%. Valores en porcentaje.

Tamaño playlist	Popularidad target	n	HitRate@10	MAP@10
long	head	6553	3.20	1.27
long	mid	5185	3.30	1.29
long	tail	8332	0.59	0.22
medium	head	10456	28.72	16.18
medium	mid	8744	22.12	12.27
medium	tail	11742	3.14	1.67
short	head	4526	28.70	16.59
short	mid	3270	24.07	12.52
short	tail	5043	3.71	1.61

7 Discusión

El sistema propuesto no domina todas las métricas, pero entrega un balance útil entre exactitud y cobertura. Esta distinción es importante para interpretar correctamente el resultado final. Si el objetivo fuera maximizar solo MAP@10 o nDCG@10, Sequential Markov sería el modelo más atractivo. Sin embargo, APC también requiere exploración, cobertura de catálogo y robustez frente a perfiles diversos. En ese escenario, el enfoque híbrido recupera un conjunto más amplio de candidatos y aumenta cobertura sin perder competitividad en HitRate@10.

La narrativa experimental muestra una trayectoria clara. Los baselines iniciales prueban que popularidad y texto del nombre de playlist contienen señal, pero son insuficientes. Los modelos fuertes muestran que estructura secuencial y colaborativa son necesarias. Finalmente, el modelo de dos etapas combina parte de esas ventajas: usa co-ocurrencia para construir candidatos musicalmente plausibles y reranking para ajustar el resultado al contexto local. La conclusión no es que un único método resuelve APC por completo, sino que la arquitectura híbrida ofrece una base reproducible y extensible para equilibrar ranking y beyond-accuracy.

La principal limitación es la ausencia de señales semánticas externas en la ejecución final. LastFM fue considerado como enriquecimiento opcional, pero se excluyó del protocolo principal por latencia y reproducibilidad limitada a escala. Esta decisión reduce riqueza de contenido, pero fortalece la validez experimental: todos los resultados pueden reproducirse localmente desde los mismos archivos. Otra limitación es que el split *leave-last-out* evalúa un único target por playlist; por tanto, las métricas capturan recuperación de una canción retenida, no satisfacción humana completa ni coherencia subjetiva de una lista extendida. Futuros trabajos podrían incorporar embeddings de audio o texto, múltiples targets por playlist y ajuste de pesos condicionado por segmento, siempre que mantengan un pipeline replicable.

8 Conclusiones

Presentamos un pipeline reproducible para APC basado en retrieval por co-ocurrencia y reranking híbrido. El modelo alcanza HitRate@10 competitivo frente a Markov, supera a ItemKNN en varias métricas y mejora la cobertura de catálogo. Las ablaciones muestran que co-ocurrencia es la señal más importante, seguida por artista y nombre de playlist. El análisis por segmentos identifica la cola larga como principal desafío futuro.

La conclusión metodológica es que la evaluación de APC debe construirse progresivamente. Los baselines simples entregan un piso necesario, los comparadores fuertes revelan qué señales son realmente competitivas y las ablaciones permiten explicar por qué el modelo final funciona. En nuestro caso, el sistema híbrido no reemplaza completamente a Markov en ranking fino, pero ofrece una solución más balanceada para escenarios donde cobertura, novedad y reproducibilidad también importan.

Repositorio

El código, notebooks ejecutados, README, resultados y figuras están disponibles en: <https://github.com/larryuc/Proyecto-RecSys-APC-Final>.

References

- [1] Andrew Mvd. Spotify Playlists. Kaggle. <https://www.kaggle.com/datasets/andrewmvd/spotify-playlists>.
- [2] Ricci, F., Rokach, L., and Shapira, B. Recommender Systems Handbook. Springer, 2015.
- [3] Schedl, M., Zamani, H., Chen, C.-W., Deldjoo, Y., and Elahi, M. Current challenges and visions in music recommender systems research. International Journal of Multimedia Information Retrieval, 2018.
- [4] Raasveldt, M. and Muhleisen, H. DuckDB: An Embeddable Analytical Database. CIDR, 2020.
- [5] Chen, C.-W. et al. RecSys Challenge 2018: Automatic Music Playlist Continuation. ACM RecSys Challenge, 2018.